

Smart MCQ Solver Challenge

Project Report

Deep Learning & Generative AI Project

Field	Details
Student Name	Rohan Kumar
Roll Number	23f3001514
Term	T2 2026
Course	Deep Learning & Generative AI
Kaggle Username	rohankumar1818
W&B Project	23f3001514-t22026

1. Problem Statement

The Smart MCQ Solver Challenge is a natural language processing competition hosted on Kaggle. The goal is to build a machine learning system that can solve complex multiple choice questions (MCQ).

Each question comes with a prompt and five answer options labeled A, B, C, D, and E. The task is to predict the top 3 most likely correct answers in ranked order for each question in the test set.

The competition uses Mean Average Precision at 3 ($\text{mAP}@3$) as the evaluation metric. This metric rewards models that rank the correct answer higher in their predictions. A correct answer at rank 1 gives a score of 1.0, at rank 2 gives 0.5, and at rank 3 gives 0.333. If the correct answer is not in the top 3, the score is 0.

Dataset: The training set contains 2000 questions with ground truth answers. The test set contains 500 questions for which predictions must be submitted.

2. Methodology

The overall pipeline follows these steps: load and explore the data, clean and preprocess the text, train multiple models with different approaches, compare their performance, and combine them using a weighted ensemble for the final prediction.

2.1 Text Preprocessing

All text was preprocessed using a `clean_text` function that converts text to lowercase, removes special characters using regular expressions, and collapses extra whitespace. This ensures consistent input to all models.

A key feature engineering step was combining the question prompt with all five answer options into a single text string. This combined text gives the model full context about both the question and all possible answers, allowing it to learn patterns of word overlap between the question and the correct answer.

2.2 Evaluation Metric — $\text{mAP}@3$

The $\text{mAP}@3$ metric was implemented from scratch and verified with a sanity check. For three test cases the result was 0.8333 which matched the expected value, confirming the implementation was correct.

The metric works by checking the rank of the correct answer in the top 3 predictions. If found at rank 1 the score is 1.0, rank 2 gives 0.5, and rank 3 gives 0.333. The final score is the mean across all questions.

3. Model Architectures

3.1 Model 1 — Logistic Regression with TF-IDF (Classical Baseline)

The first model uses TF-IDF (Term Frequency Inverse Document Frequency) to convert the combined text into numerical features. TF-IDF gives high importance to words that appear

frequently in one document but rarely across all documents, making them discriminative features.

A TfidfVectorizer with max_features=5000 was fitted on the training data and used to create feature matrices. Only 2940 features were actually created, which means the entire dataset vocabulary is small and domain-specific.

Logistic Regression was trained on these features with C=1.0 regularization, lbfgs solver, and multinomial multi-class setting. Five-fold cross validation was used to get honest performance estimates.

Hyperparameter	Value	Reason
max_features	5000	Top 5000 most important words
C	1.0	Default regularization — no overfitting penalty
solver	lbfgs	Efficient for small datasets
multi_class	multinomial	Predicts across all 5 classes at once
max_iter	1000	Enough steps for convergence
cv_folds	5	Honest cross-validation estimate

3.2 Model 2 — TextCNN (Built From Scratch)

The second model is a Convolutional Neural Network for text classification, built entirely from scratch using PyTorch. This model learns to detect patterns in text using sliding convolutional filters.

The architecture uses an embedding layer that converts word indices to 128-dimensional dense vectors. Three separate convolutional filters with kernel sizes 2, 3, and 4 slide over the embedded text to capture 2-word, 3-word, and 4-word patterns respectively. Global max pooling keeps the most important feature from each filter regardless of where in the text it appears. Dropout at 50% prevents overfitting. The final fully connected layer maps 384 features (3×128) to 5 class scores.

Component	Configuration	Purpose
Embedding	2975 words * 128 dims	Word to vector conversion
Conv2	128 filters, kernel=2	Bigram pattern detection
Conv3	128 filters, kernel=3	Trigram pattern detection
Conv4	128 filters, kernel=4	4-gram pattern detection
MaxPool	Global max pooling	Keep most important feature
Dropout	p=0.5	Prevent overfitting
FC Layer	384 => 5	Final classification
Total Params	530,565	Mostly in embedding layer

3.3 Model 3 — TextCRNN / CNN + BiLSTM (Built From Scratch)

The third model combines CNN and Bidirectional LSTM, also built from scratch using PyTorch. CNN captures local patterns while BiLSTM understands the full sequence context by reading text both forward and backward.

The architecture starts with an embedding layer identical to TextCNN. A single CNN layer with kernel size 3 and padding 1 extracts local features while keeping the sequence length unchanged. A 2-layer Bidirectional LSTM then reads the CNN output in both directions. The

final hidden states from the forward and backward passes are concatenated to form a 256-dimensional representation which is mapped to 5 class scores.

Component	Configuration	Purpose
Embedding	2975 words $\times$ 128 dims	Word to vector conversion
Conv1D	128 filters, kernel=3, padding=1	Local feature extraction
BiLSTM	128 hidden, 2 layers, bidirectional	Sequential context understanding
Dropout	p=0.5	Prevent overfitting
FC Layer	256 $\rightarrow$ 5	Final classification
Total Params	1,090,821	More complex than TextCNN

3.4 Model 4 — SBERT Pretrained Transformer (HuggingFace)

The fourth model uses a pretrained transformer model from HuggingFace called all-MiniLM-L6-v2. This is a BERT-based model that was already trained by HuggingFace on over 1 billion sentence pairs for semantic similarity tasks.

The model was not trained or fine-tuned on our data. Instead it was used directly to convert sentences into 384-dimensional embedding vectors. For each question, the prompt was encoded into a vector and each of the 5 options was also encoded separately. Cosine similarity between the prompt vector and each option vector was computed to determine which option is most semantically similar to the question. Softmax was applied to convert similarities into probabilities.

This approach is called zero-shot inference because the model generalizes to our task without any task-specific training. The result is a probability distribution over the 5 options for each question.

Property	Value
Model Name	sentence-transformers/all-MiniLM-L6-v2
Architecture	BERT-based, 6 transformer layers
Embedding Dimension	384
Training Data	1 billion sentence pairs
Strategy	Cosine similarity between prompt and options
Training Required	No — pretrained and used directly

4. Training Details

4.1 Data Split

The 2000 training samples were split into 80% training (1600 samples) and 20% validation (400 samples) for the deep learning models. The test set of 500 samples was used only for final predictions.

4.2 Tokenization

A custom vocabulary was built from all words appearing at least twice across the training and test data combined. The final vocabulary size was 2975 words — much smaller than the set limit of 15000, confirming the dataset is domain-specific with limited unique terminology.

All sequences were padded or truncated to a maximum length of 256 tokens. The <PAD> token at index 0 fills shorter sequences. The <UNK> token at index 1 handles words not seen during vocabulary building.

4.3 Training Loop

Both TextCNN and TextCRNN were trained using the same training loop with the following setup:

- Loss function: CrossEntropyLoss for multi-class classification
- Optimizer: Adam with learning rate 0.001 and weight decay 1e-4
- Learning rate scheduler: ReduceLROnPlateau — reduces LR by 50% if validation accuracy does not improve for 3 epochs
- Gradient clipping: max_norm=1.0 to prevent exploding gradients
- Early stopping: patience of 5 epochs — stops training if no improvement
- Best model checkpointing: saves weights when validation accuracy improves
- Batch size: 32 — good balance between memory and stability

4.4 Training Results

Model	Epochs Run	Best Val Accuracy	Best Val F1	Early Stop?
TextCNN	7	1.0000	1.0000	Yes at epoch 7
TextCRNN	16	1.0000	1.0000	Yes at epoch 16

Both models achieved 100% validation accuracy. This is because with only 2000 training samples and a small vocabulary of 2975 words, the models memorize the training patterns. The more honest evaluation is the Kaggle leaderboard score on unseen test data.

TextCRNN took longer to converge (16 epochs vs 7 for TextCNN) because it has more parameters and a more complex architecture with BiLSTM that requires more iterations to learn.

5. Experiment Tracking with Weights & Biases

All experiments were tracked using Weights & Biases (W&B). Five separate runs were logged to the project 23f3001514-t22026, allowing comparison of all models on the same metrics dashboard.

W&B Run Name	Model	Metrics Logged
logistic-regression-tfidf	Logistic Regression	macro_f1, accuracy, map_at_3
textcnn-scratch	TextCNN	train_loss, train_acc, val_acc, val_macro_f1 per epoch
textcrnn-cnn-lstm	TextCRNN	train_loss, train_acc, val_acc, val_macro_f1 per epoch
sbert-all-minilm-l6-v2	SBERT Pretrained	macro_f1, accuracy, map_at_3
model-comparison-summary	All Models	Comparison chart with all model accuracies

6. Evaluation Results

6.1 Model Performance Comparison

Model	CV Macro F1	CV Accuracy	Train mAP@3	Train Accuracy
Logistic Regression	0.9987	0.9985	1.0000	1.0000
TextCNN	-	-	-	1.0000
TextCRNN	-	-	-	1.0000
SBERT Pretrained	0.2587	0.2610	0.4231	0.2610
Final Ensemble	-	-	-	-

Note: CV (cross validation) scores were calculated for Logistic Regression. TextCNN and TextCRNN were evaluated on the held-out validation split. SBERT used cosine similarity without any training so its scores reflect its zero-shot performance.

6.2 Kaggle Leaderboard Score

Submission	Score	Status
Best submission	0.75810	Above cutoff
Required cutoff	0.73000	Minimum to qualify for viva

7. Error Analysis

7.1 Training Set Error Analysis

Model	Correct	Wrong	Train Accuracy
Logistic Regression	2000	0	1.0000
TextCNN	2000	0	1.0000
TextCRNN	2000	0	1.0000

All three models achieved 100% accuracy on the training data. This is a sign of overfitting — the models have memorized the training examples. However this is expected behaviour with a small dataset of 2000 samples and a limited vocabulary.

7.2 Key Observations

- All 3 models got all 2000 training questions correct — no hard cases found in training data
- Zero questions were wrong for all 3 models simultaneously — every question had at least one model predict correctly
- The gap between train accuracy (100%) and Kaggle test score (75.27%) clearly shows overfitting
- SBERT underperformed significantly — mAP@3 of 0.4231 vs LR mAP@3 of 1.0 on training data
- SBERT struggles because all 5 options are semantically similar long paragraphs — cosine similarity cannot distinguish the correct answer

7.3 Why Models Overfit

The training dataset contains only 2000 questions with a vocabulary of just 2975 unique words. The correct answer in many questions repeats key terminology from the question prompt. TF-IDF and deep learning models both learn this lexical overlap pattern very well on training data, but the test set has more varied question types that require genuine reasoning rather than word matching.

8. Final Ensemble

The final submission combines predictions from all three locally trained models using a weighted average of their probability distributions. SBERT was excluded from the ensemble because its training data performance (42.31% mAP@3) was much lower than the other models.

Model	Weight	Reason
Logistic Regression	0.80 (80%)	Strongest performer — TF-IDF captures lexical overlap effectively
TextCNN	0.10 (10%)	Adds diversity through local pattern detection
TextCRNN	0.10 (10%)	Adds diversity through sequential context understanding

For each test question, the weighted average probability across 5 options was computed. Options were sorted from highest to lowest probability. The top 3 options were joined as a space-separated string to form the submission prediction.

Validation checks were performed before saving: correct row count (500), no missing predictions, and exactly 3 predictions per row. The submission was saved to /kaggle/working/submission.csv.

9. Key Insights

9.1 Why Logistic Regression Outperforms Deep Learning

This was the most surprising finding of the project. A simple Logistic Regression model outperformed both TextCNN and TextCRNN in the final ensemble weight allocation. The reason is that the correct answer in MCQ questions tends to repeat key words and phrases from the question prompt. TF-IDF is specifically designed to capture this kind of word importance and overlap. With only 2000 training samples, deep learning models do not have enough data to learn more complex reasoning patterns.

9.2 Why SBERT Underperformed

SBERT converts text to semantic vectors and computes cosine similarity. This approach assumes the correct answer will be semantically closest to the question. However in this dataset all five options are long detailed paragraphs about the same topic — they are all semantically similar to the question. SBERT cannot distinguish which one is specifically the correct answer because the semantic distance between all options is very small.

9.3 Dataset Characteristics

The vocabulary of only 2975 unique words across 2500 questions confirms the dataset is highly domain-specific. All questions come from similar academic topics using the same terminology repeatedly. This explains why a simple TF-IDF approach works so well — the signal is concentrated in a small set of important domain-specific words.

9.4 Overfitting Analysis

All models show a significant gap between training performance (100% accuracy) and test performance (75.27% mAP@3). This gap indicates the models are memorizing training patterns rather than learning to reason. The primary cause is the small training set size of 2000 questions. More training data or fine-tuning a pretrained language model would likely reduce this gap.

10. Future Work and Improvements

- Fine-tune a pretrained transformer model such as BERT or RoBERTa specifically on this MCQ dataset using LoRA for parameter-efficient training
- Implement a RAG pipeline that retrieves similar training questions as context before making predictions
- Collect or augment training data to reduce overfitting — more data would help deep learning models generalize better
- Use cross-encoder models for more accurate relevance scoring between questions and options
- Experiment with larger ensemble combinations including SBERT after improving its accuracy through fine-tuning
- Apply temperature scaling to calibrate model probabilities before ensembling for better combination

11. Tools and Technologies

Tool	Purpose
Kaggle	Competition platform, GPU training (Tesla T4), submission
GitHub	Version control, code history, project organization
Weights & Biases	Experiment tracking, metric logging, model comparison
PyTorch 2.10	Building TextCNN and TextCRNN from scratch
scikit-learn	Logistic Regression, TF-IDF, cross-validation, metrics
sentence-transformers	Pretrained SBERT model for semantic embeddings
HuggingFace Transformers	Pretrained model loading and zero-shot inference
Chonkie	Text chunking for RAG pipeline exploration
NumPy / Pandas	Data manipulation and numerical computation
Matplotlib	EDA plots, model comparison charts

12. Conclusion

This project successfully built a multi-model pipeline for solving multiple choice questions achieving a Kaggle mAP@3 score of 0.75270, which is above the required cutoff of 0.73.

Four different models were implemented and evaluated: a classical Logistic Regression baseline, two deep learning models built from scratch (TextCNN and TextCRNN), and a pretrained transformer (SBERT). All experiments were tracked with Weights & Biases for proper comparison.

The key finding was that Logistic Regression with TF-IDF features significantly outperformed more complex models on this dataset. This demonstrates that the choice of model should be driven by data characteristics rather than model complexity. With limited training data and domain-specific vocabulary, simple models can outperform deep learning.

The final weighted ensemble of all three locally trained models achieved the best Kaggle performance. Future improvements involving fine-tuned transformers and RAG pipelines are expected to push the score higher.